

Frontend Architecture & Design Reference

Every page, component, route, and business rule — one place

CONTENTS

1. Architecture Overview
 2. Project Structure
 3. Routing & Access Control
 4. Global State — Auth Store
 5. Public Pages
 6. Citizen Pages
 7. Staff Pages
 8. Admin Pages
 9. Shared Components
 10. The Mock API Layer (Offline Backend)
 11. Key Business Rules Reference
-

1. Architecture Overview

Stack: Vue 3 (`<script setup>`), Vue Router 4, Pinia (auth store), Vite. No CSS framework — a single hand-written stylesheet (`assets/main.css`) plus per-component `<style scoped>` blocks.

The most important pattern in this codebase: every Vue page imports a single module, `src/api/index.js`, and calls it exactly like axios:

```
import api from "../../api"
const { data } = await api.get("/requests")
await api.post("/requests", payload)
```

Right now, `src/api/index.js` is **not** a thin HTTP client — it's a full offline mock backend. It intercepts every call, matches the URL/method, and returns fake data from `src/api/seed.js`, fully replicating whatever the real Flask backend is supposed to do (validation, role checks, business rules — all of it). This means:

- The frontend runs completely standalone, with zero network calls to a real server.
- A frontend developer can build an entire new feature — new page + new mock route handler + seed data — without touching the backend at all.
- When a real backend is ready, swapping to it is a one-file change (point the API base URL at the Flask server instead of importing the mock) — no Vue page needs to change, **as long as the real backend's responses match the same shape the mock already returns**.

Implication for the team: `seed.js` + `index.js` are the closest thing this project has to an API spec. Whoever builds the real backend should treat the mock's current behavior as the contract to replicate exactly (see Section 10).

2. Project Structure

```
frontend/src/
├─ App.vue           // root: just <router-view />
├─ main.js           // mounts app, installs Pinia + Router
├─ api/
│   ├─ index.js      // mock "backend" – routes + business logic
│   └─ seed.js       // fake data: users, requests, facilities, ...
├─ stores/
│   └─ auth.js       // Pinia store: token, user, login/register/logout
├─ router/
│   └─ index.js      // route table + role-based navigation guard
├─ utils/
│   └─ status.js     // fmtStatus() – badge label formatting
├─ components/       // shared, reused across many pages
├─ views/            // public pages (Landing, Login, Register, ...)
└─ views/citizen/    // citizen-only pages
```

```
|— views/staff/           // staff-only pages
|— views/admin/           // admin-only pages
```

3. Routing & Access Control

Defined in `router/index.js`. A global `router.beforeEach` guard: public routes always pass; everything else requires login; if a route has `meta.roles` and the user's role isn't in that list, they're redirected to their own role's dashboard.

PATH	ROLE	PATH	ROLE
/	public	/dashboard	citizen
/login	public	/submit	citizen
/register	public	/complaints, /complaints/:id	citizen
/forgot-password	public	/facilities, /bookings, /payments	citizen
/track	public	/subscription	citizen
/feed, /announcements, /notifications	all roles	/staff/dashboard, /staff/complaints(/:id)	staff
/admin/dashboard	admin	/staff/community	staff
/admin/departments, /officers	admin	/admin/complaints	admin
/admin/facilities, /bookings	admin	/admin/subscriptions	admin
/admin/users, /community, /reports	admin	/:pathMatch(.*)*	redirects to /

4. Global State — Auth Store (`stores/auth.js`)

STATE

`token`, `user` — both persisted to `localStorage` (`civic_token`, `civic_user`)

COMPUTED

`isLoggedIn`, `role`, `isCitizen`, `isStaff`, `isAdmin`

ACTIONS

- `login(email, password)` → `POST /auth/login`
- `register(payload)` → `POST /auth/register` (always creates a citizen)
- `logout()` — local only, clears state + `localStorage`, no API call
- `refreshMe()` → `GET /auth/me`, refreshes cached user

5. Public Pages

Landing `views/LandingView.vue` `/`

- Marketing homepage: navbar, hero with CTAs ("Submit a Complaint" → register, "Track Complaint" → /track), 4-step "How It Works", live stats strip, footer
- Stats pulled from `GET /public-stats` (total complaints, resolved, in progress, departments)
- No business logic — purely presentational

Login `views/LoginView.vue` `/LOGIN`

- Email/password form, password-visibility toggle, clickable "Demo Credentials" box that autofills Admin/Officer/Citizen logins
- Only native `required` validation — no client-side format checks
- Redirects by role after login: citizen → `/dashboard`, staff → `/staff/dashboard`, admin → `/admin/dashboard`

Register `views/RegisterView.vue` `/REGISTER`

- Name / email / password form — always creates a **citizen** account
- Name must match `^[A-Za-z][A-Za-z\s.'-]{1,79}$` (letters only — rejects digits); password ≥ 6 characters
- Redirects to `/dashboard` on success

Forgot Password `views/ForgotPasswordView.vue` `DEMO ONLY`

- **Not backend-integrated** — entirely simulated with `setTimeout`, no `api` calls at all
- "Send OTP" shows a fake message ("Demo: Use OTP 123456"); "Reset Password" just checks the two password fields match — no OTP is actually verified

Track Complaint `views/citizen/TrackComplaint.vue` **PUBLIC**

- Public lookup by Complaint ID (e.g. `CMP0007`) — no login required
- Calls `GET /track?id=...`, shows status/timeline for that one complaint (citizen PII excluded server-side)

6. Citizen Pages

Dashboard `views/citizen/CitizenDashboard.vue` `/DASHBOARD`

- Greeting header, 4 stat cards (Total/Pending/In Progress/Resolved), Recent Complaints table (last 5)
- Premium subscription banner: shown only if not subscribed or has a payment due (fetched separately from the main dashboard load so a failure here never blocks the rest of the page)
- Calls: `GET /dashboard`, `GET /requests`, `GET /notifications`, `GET /subscriptions/me`

Submit Complaint `views/citizen/NewRequest.vue` `/SUBMIT`

- **Requires an active, paid-up Premium subscription** — shows a locked card instead of the form otherwise (see Section 11)
- Form: category, AI-suggested category (`POST /ai/suggest`), paper-scan-to-form (`POST /ai/extract-from-image`), title, address, map pin (`LocationPicker`), ward, description, priority, photos (`MultiImageUpload`, max 3)
- Before creating, checks for likely duplicates (`POST /requests/similar`) and shows `SimilarComplaints` so the citizen can upvote an existing one instead
- Creates via `POST /requests`

My Complaints `views/citizen/MyRequests.vue` `/COMPLAINTS`

- List + filter of the citizen's own complaints; "Delete" action shown only while `status === 'pending'`
- Calls: `GET /requests`, `DELETE /requests/:id`

Complaint Detail `views/citizen/ComplaintDetail.vue` `/COMPLAINTS/:ID`

- Full detail + timeline (Submitted → Assigned → In Progress → Resolved), photos/evidence via `ImageGallery`
- "Delete Complaint" button shown only while `status === 'pending'` — redirects to `/complaints` after
- Embeds `ResolveActions` when resolved/closed/reopened, for citizen close-with-rating or reopen
- Calls: `GET /requests/:id`, `DELETE /requests/:id`

Facilities / Bookings / Payments views/citizen/FacilitiesView.vue, MyBookings.vue, MyPayments.vue

- Browse bookable facilities → book a date/time slot (validated against capacity, past dates, and overlapping bookings) → pay the generated fee → view invoice
- My Bookings: cancel pending bookings, view/download invoice; My Payments: pay pending dues, full transaction history
- Calls span: `GET /facilities`, `GET /facilities/:id/availability`, `POST /bookings`, `DELETE /bookings/:id`, `GET /bookings/:id/invoice`, `GET /payments`, `POST /payments/:id/pay`

My Subscription views/citizen/MySubscription.vue **/SUBSCRIPTION**

- Cyber Panchayat Premium (₹100/month) — subscribe / cancel / resume / pay-due / view invoice; full billing history
- See Section 11 for the underlying business rules (paywall, grace period, late-payment handling)
- Calls: `GET /subscriptions/me`, `POST /subscriptions/subscribe`, `POST /subscriptions/cancel`, `GET /subscriptions/payments`, `POST /subscriptions/payments/:id/pay`, `GET /subscriptions/payments/:id/invoice`

Community Feed views/citizen/CommunityFeed.vue **/FEED**

- Instagram-style feed: create post (caption + photos), like, comment, delete own post/comment; "All Posts" / "My Posts" tabs
- **Requires an active, paid-up Premium subscription** to post/like/comment — viewing the feed stays free (see Section 11)
- Calls: `GET /posts`, `POST /posts`, `DELETE /posts/:id`, `POST /posts/:id/like`, `GET / POST /posts/:id/comments`, `DELETE /posts/:id/comments/:id`

Announcements & Notifications views/AnnouncementsView.vue, NotificationsView.vue **SHARED, ALL ROLES**

- Announcements: read-only for citizens/staff, admin can post/delete (`category: 'announcement'`)
- Notifications: full inbox, mark one or all as read (`PUT /notifications/0/read` is the "mark all" sentinel)

7. Staff Pages

Staff Dashboard `views/staff/StaffDashboard.vue` `/STAFF/DASHBOARD`

- Stat cards (Total Assigned/Pending/In Progress/Resolved) + Recent Assigned Complaints table

My Assignments `views/staff/MyAssignments.vue` `/STAFF/COMPLAINTS`

- All complaints assigned to this staff member — search, status filter, pagination (client-side)

Update Status `views/staff/UpdateStatus.vue` `/STAFF/COMPLAINTS/:ID`

- Move a complaint to `in_progress` or `resolved` (with evidence photos), add remarks
- Form is replaced by an info alert when the complaint is terminal for staff: `closed`, `on_hold_weather` (needs admin), or already `resolved`

Community Posts (Staff) `views/staff/StaffCommunityPosts.vue` `/STAFF/COMMUNITY`

- Same feed as citizens, plus posting as an official department voice — staff/admin posts and comments are always flagged `is_official`

8. Admin Pages

Admin Dashboard `views/admin/AdminDashboard.vue` **/ADMIN/DASHBOARD**

- City-wide stat cards, complaints-by-status donut chart, citizen satisfaction average, Premium Subscribers stat, Reports & Analytics charts (complaints over time, department performance) merged into the same page

Manage Departments `views/admin/ManageDepartments.vue`

- CRUD for departments; officers-per-department shown via client-side name matching (not by ID)

Manage Officers `views/admin/ManageOfficers.vue`

- CRUD for staff accounts; name/phone regex validation matches Register; average citizen rating computed client-side per officer
- Deactivating an officer with open complaints shows a warning to reassign them first

Manage Complaints `views/admin/ManageRequests.vue`

- Central console: assign/reassign officer (grouped by department), weather-hold/resume, budget-fund tagging, reject (pending-only, reason required)

Manage Facilities / Bookings `views/admin/ManageFacilities.vue, ManageBookings.vue`

- Facility CRUD (fixed type/ward lists) + booking status management (confirm/cancel/complete)

Manage Subscriptions `views/admin/ManageSubscriptions.vue`

- Every citizen's subscription status, billing dates, last invoice; MRR / collected / pending stats

Manage Users, Community, Reports

- `ManageUsers.vue` — role/ward edit, enable/disable, subscription badge per citizen
- `CommunityManagement.vue` — moderation over all posts
- `ReportsAnalytics.vue` — merged into Admin Dashboard, also available standalone

9. Shared Components

AppSidebar / AppTopbar

- Sidebar: brand, role-based nav tree, user card, logout. Topbar: page title bar only (notifications now live in the sidebar)

ImageGallery

- Responsive grid, caps at 3 images regardless of input length

LocationPicker

- Leaflet map, geolocation, reverse-geocode via Nominatim (best-effort), emits `{lat, lng, address}`

MultImageUpload

- 5MB/file cap, configurable max count (default 3), uploads via `POST /upload` one file at a time

NotificationsPanel

- Slide-in flyout alternative to the full Notifications page

ResolveActions

- Citizen close-with-rating (1-5 stars, required) / reopen (capped at 2 times)

SimilarComplaints

- Duplicate-prevention: upvote an existing complaint instead of filing a new one

10. The Mock API Layer (Offline Backend)

`src/api/index.js` exports an axios-compatible object: `{ get, post, put, delete, patch }`. Every call goes through a single `handle(method, url, body)` router that pattern-matches the URL and returns `{ data: ... }` or throws `{ response: { status, data } }` to simulate an HTTP error — exactly how a real axios error looks, so page-level `try/catch` code doesn't know the difference.

`src/api/seed.js` holds every fake data array (`USERS`, `REQUESTS`, `FACILITIES`, `BOOKINGS`, `PAYMENTS`, `POSTS`, `SUBSCRIPTIONS`, etc.) as plain JS objects — these mutate in memory as the app is used, and reset on a full page reload.

For whoever adds a new feature to the frontend: add your fake data to `seed.js`, add a route branch to `handle()` in `index.js` matching the method + path your Vue page will call, and return data shaped exactly like your Vue page expects. Don't touch other people's existing route branches or seed arrays.

For whoever builds the real backend later: the mock's route branches are effectively already-tested pseudocode for the real Flask endpoints — same URL, same method, same request/response shape, same validation rules. Match it exactly rather than redesigning from scratch.

11. Key Business Rules Reference

Complaints

Delete own complaint	Citizen only, only while <code>status === 'pending'</code>
Reject a complaint	Admin only, only while <code>pending</code> , reason required
Reopen after resolved	Citizen only, capped at 2 reopens total
Terminal states for staff	<code>closed</code> , <code>on_hold_weather</code> , <code>resolved</code> , <code>rejected</code> all block further staff updates

Facility Bookings

Booking window	06:00–22:00 only, must be a future date/time
Conflict check	Overlapping time slot on the same facility/date is rejected (409)
Cancel booking	Only while <code>status === 'pending'</code> ; paid+cancelled bookings auto-refund

Premium Subscription (₹100/month)

Paywall	Filing a complaint or posting/liking/commenting on the community feed requires an active AND paid-up subscription — an unpaid due blocks access even if status is "active"
Cancel — fully paid up	Stays active through the period already paid for ("Ending Soon"); auto-closes for real once that period ends
Cancel — unpaid due exists	Ends immediately; the unpaid invoice is waived
Resume	Undo a pending cancellation any time before the paid period ends — no new charge
Late payment	Paying an overdue invoice shifts that cycle to start <i>today</i> , so the citizen always gets a full paid month from the day they actually pay
Grandfather clause	Complaints/posts made while subscribed remain valid forever, even after the subscription later lapses

Accounts

Name validation	<code>^[A-Za-z][A-Za-z\s\.'-]{1,79}\$</code> everywhere (Register, Manage Officers) — letters only
Phone validation	<code>^[6-9]\d{9}\$</code> — 10-digit Indian mobile
Password	Minimum 6 characters